Transport & Tour Booking System (BusBook Pro)

Project: Software Engineering (DLMCSPSE01)

Student Name: Kruthik Gowda Halebeedu Ravi

Matriculation No: 42310494

Date: 11/03/2026

Table of Contents

List of Figures: 3

1. Introduction 4

2. Project Profile 5
   - 2.1 Project Objectives 5
   - 2.2 Target Users 5
   - 2.3 project Scope 5

3. Software Development Methodology 6

4. Requirements Analysis 6
   - 4.1 Requirements Analysis 6
   - 4.2 Non-Functional Requirements 7
   - 4.3 Glossary 7

5. System Architecture 8

6. Technology stack 9

7. Project Plan 10

8. Database Design 11

9. System Implementation 13

10. Testing and Quality Assurance 16

11. Project Risks 16

12. Lessons Learned 17

13. Future Improvements 17

14. Conclusion 18

List of Figures:

Figure 1: System Architecture…….07

Figure 2: Project Development timeline……10

Figure 3: Database Entity relationship diagram ……12

Figure 4: Homepage of the bus and tour booking system……13

Figure 5: User Login Interface……14

Figure 6: User Registration Page……14

Figure 7: User Dashboard……15

Figure 8: Admin Dashboard……15

1. Introduction

Digital transformation has significantly changed the way transportation and tourism services are managed. Traditional booking system often relies on manual processes or fragmented digital tools that may lead to inefficiencies, booking conflicts, and limited accessibility for customers. With the growing demand for convenient and reliable online services, web-based booking platforms have become essential for improving operational efficiency and customer experience.

This project presents the design and development of BusBooking Pro, a web-based transport and tour booking systems. The goal of the system is to provide users with an easy and reliable way to search for transport routes, book seats, and manage reservations while enabling administrators to manage routes, tours, and customer data through an integrated management interface.

The application is designed as full-stack web system that integrates a responsive user interface, a backend application layer, and a relational database. The project follows software engineering principles, including requirements analysis, system architecture design, implementation, and testing.

The main objective of this project are:

- To design a functional web application for transport and tour booking
- To implement a secure and scalable booking management system
- To demonstrate the application of software engineering methodologies
- To ensure the reliability and usability of the developed system

The system aims to deliver practical value to passenger by simplifying the booking process while providing administrators with efficient tools for system management.

2. Project Profile

2.1 Project Objectives

The primary objective of the project is to develop a web-based booking platform that allows users to browse the available transport routes and tour packages and reserve seats online.

Specific objectives include:

- Providing an easy-to-use interface for searching bus routes and tour packages.
- Allowing users to create accounts and securely log into the system.
- Enabling users to book transportation and tour packages online.
- Providing administrators with tools to manage routes, tours, users, and bookings.
- Improving booking efficiency and reducing manual administrative tasks.

2.2 Target Users

The application targets two main groups of users:

Passengers

Passengers use the system to:

- Register and create accounts
- Search available routes and tours
- Booking seats
- Manage booking history

Administrators manage the system and perform tasks such as:

- Managing routes and tour packages
- Viewing booking data
- Managing user accounts

2.3 project Scope

the system focuses on core booking functionality including user registration, route browsing, and reservation management. Advanced features such as payment integration and automated notifications are considered future enhancements.

3. Software Development Methodology

This project follows an Agile-inspired development approach that emphasizes iterative development and continuous improvement.

The main phases of the methodology include:

1. Requirements analysis
2. System design
3. Implementation
4. Testing
5. Deployment

Agile methodology was chosen because it allows the system to be developed incrementally. Each component can be implemented and tested individually, making it easier to identify issues early in the development process.

4. Requirements Analysis

4.1 Requirements Analysis

Functional requirements describe the main operations the system must support.

ID	Requirement
FR1	Users must be able to register an account
FR2	Users must be able to log in securely
FR3	Users must be able to browse available routes
FR4	Users must be able to view tour packages
FR5	Users must be able to book seats
FR6	Users must be able to view booking history
FR7	Administrators must be able to manage routes
FR8	Administrators must be able to manage tours

FR9	Administrators must be able to manage users
FR10	Administrators must be able to view booking information

4.2 Non-Functional Requirements

Non-functional requirements describe system performance, security, and usability characteristics.

ID	Requirements
NFR1	System response time should be less than 3 seconds
NFR2	Passwords must be encrypted using hashing
NFR3	The system must prevent SQL injection attacks
NFR4	The application must maintain database integrity
NFR5	The system must maintain database integrity
NFR6	The system must ensure secure session management

These requirements ensure the system remains secure, reliable, and efficient.

4.3 Glossary

Term	Description
Booking	Reservation of a seat or tour
Route	Travel path between locations
Tour Package	Organized travel service
Admin	System administrator responsible for management

5. System Architecture

The system follows a three-tier architecture, which separates the application into three logical layers.

Presentation Layer

The presentation layer consists of the user interface developed using **HTML, CSS, and JavaScript**. It allows users to interact with the system through web pages.

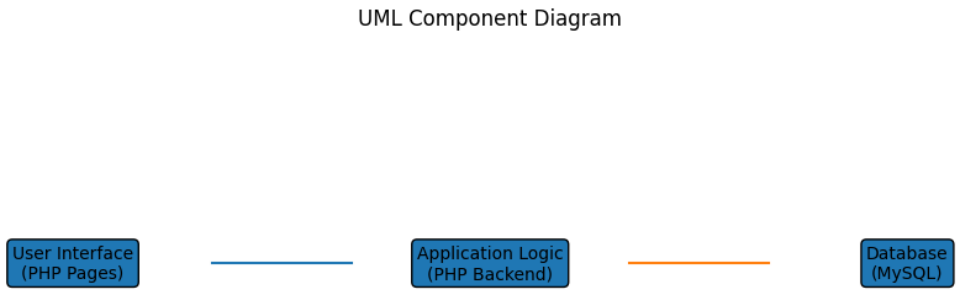
Application Layer

The application layer contains the business logic implemented in PHP. It processes user requests, perform validation, and communicates with the database.

Data Layer

The data layer is implemented using MySQL, which stores user data, route information, tour packages, and booking records.

This architecture improves system maintainability and scalability by separating different functional responsibilities.



Example architecture representation:

User Interface (HTML/CSS/PHP)

|

Application Logic (PHP Backend)

|

Database (MySQL)

Figure 1: System Architecture

6. Technology stack

The system was implemented using the following technologies:

HTML and CSS

Used to design the user interface and structure the web pages.

JavaScript

Used to add interactivity and improve user experience.

PHP

Used for server-side processing, user authentication, and booking logic.

MySQL

Used as the relational database management system.

XAMPP

Used as a local development environment that integrates Apache, PHP, and MySQL.

These technologies were selected because they are widely used in web development and provide reliable performance for full-stack applications.

7. Project Plan

The development of the system followed a structured timeline. Each stage of the project focused on a specific development activity.

Phase	Activity	Duration
Phase 1	Requirement Analysis	Week 1
Phase 2	System Design	Week 2
Phase 3	Implementation	Week 3
Phase 4	Testing	Week 4
Phase 5	Deployment	Week 5

The development timeline is illustrated in the following Gantt chart.

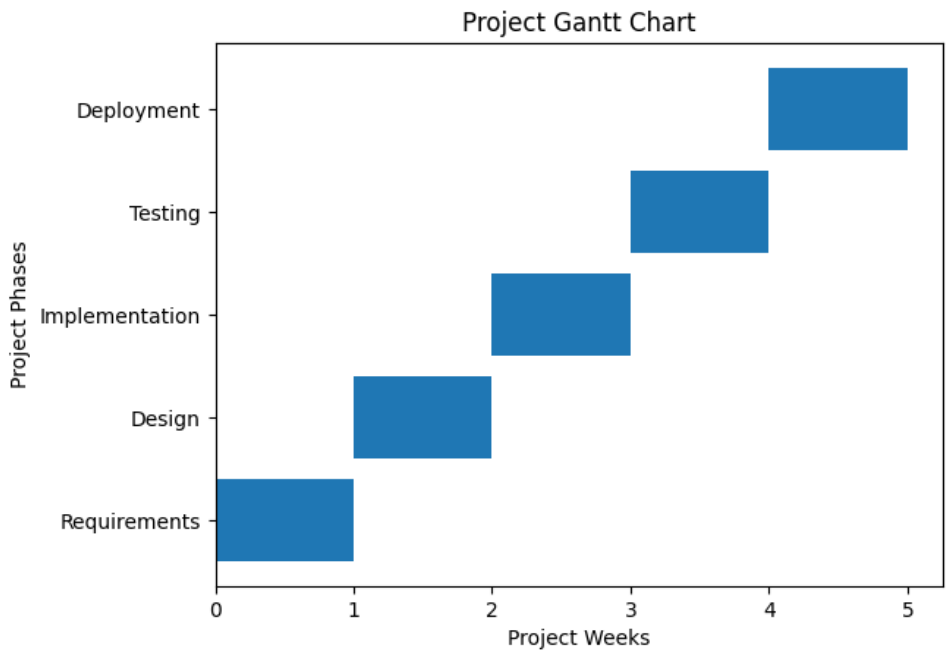


Figure 2: Project Development timeline

This plan allowed the project to progress systematically while ensuring that each phase was completed before moving to the next stage.

8. Database Design

The database design ensure that data is stored efficiently and relationships between entities are maintained.

The main entities in the system included:

- Users – stores user account information
- Routes – stores bus route information
- Tours – stores tour package information
- Bookings – stores booking transactions

Relationships between the tables include:

- One user can have multiple bookings.
- A route can be booked by multiple users.
- A tour package can be booked by multiple usres.

Users Table

Field	Description
user_id	Primary key
email	User email
password_hash	Encrypted password
role	User role

Routes Table

Field	Description
route_id	Primary key
source	Departure location
destination	Arrival location
price	Ticket price

Tours Table

Field	Description
tour_id	Primary key
tour_name	Name of tour
price	Tour price
capacity	Maximum seats

Bookings Table

Field	Description
booking_id	Primary key
user_id	Foreign key referencing users
route_id	Foreign key referencing routes
tour_id	Foreign key referencing tours

Relationships between tables are enforced using foreign keys to maintain data integrity.

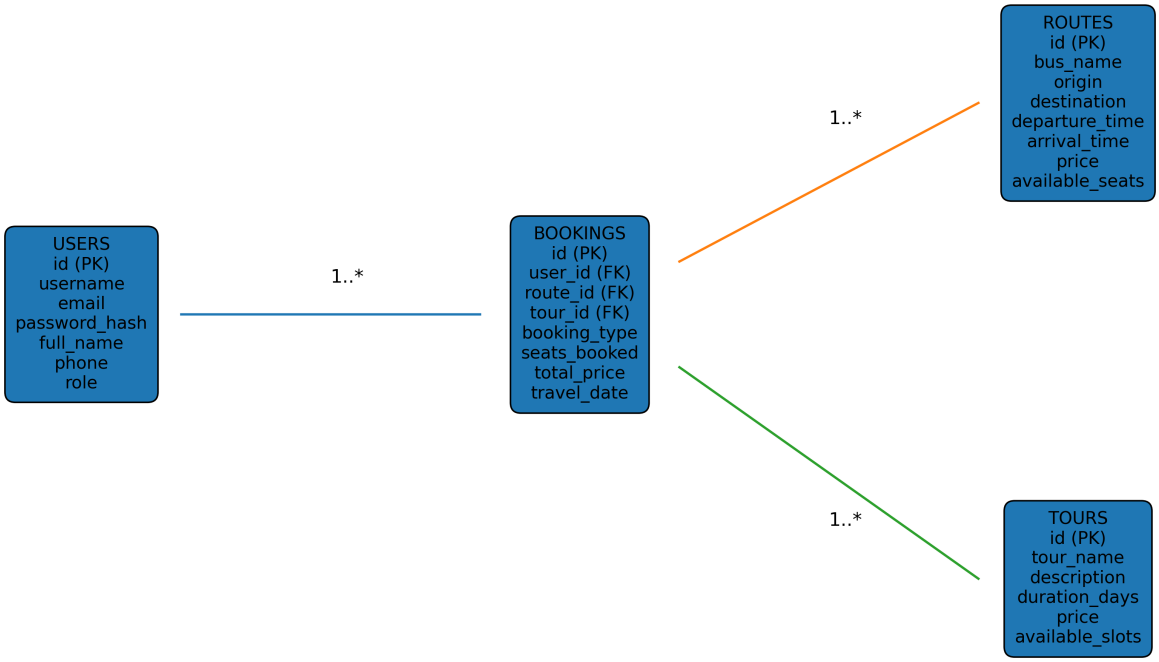


Figure 3: Database Entity relationship diagram

9. System Implementation

The system was implemented using a full-stack web development approach.

The main system modules include:

- **User module**
  Allows user to register, log in, and manage their bookings.
- **Route module**
  Displays available bus routes and allows users to book transportation.
- **Tour module**
  Display available tour packages and allows users to book tours.
- **Booking module**
  Handles booking operation and stores booking data.
- **Admin module**
  Provides administrative tools for managing routes, tours, users, and booking.

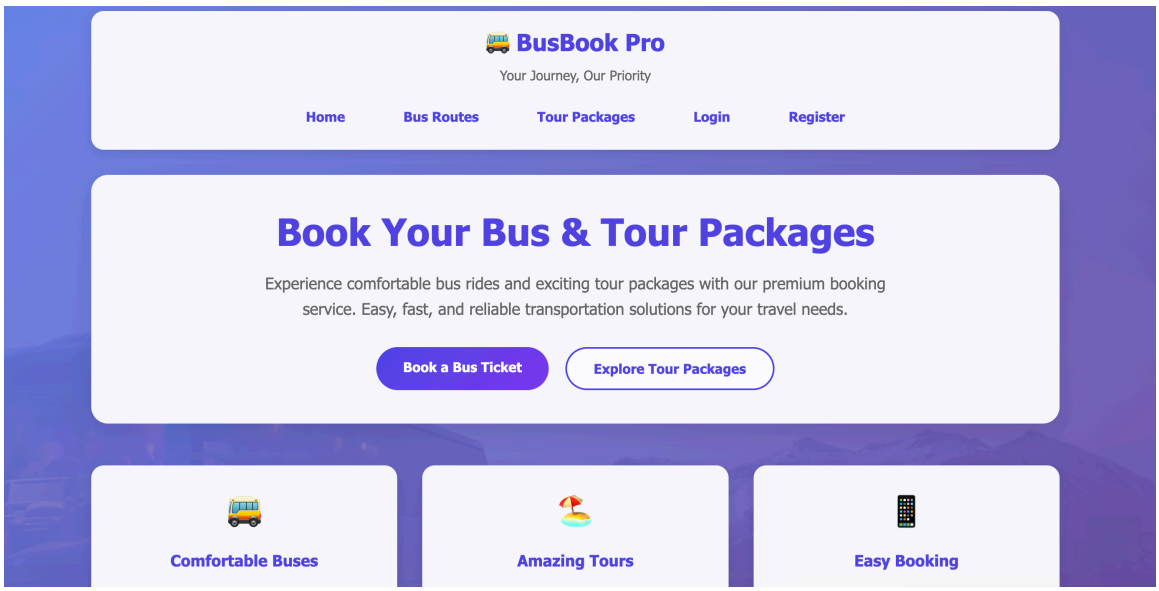


Figure 4: Homepage of the bus and tour booking system.

The home page provides an overview of the booking platform and allows users to navigate to different section of the system such as routes, tours, login, registration pages.

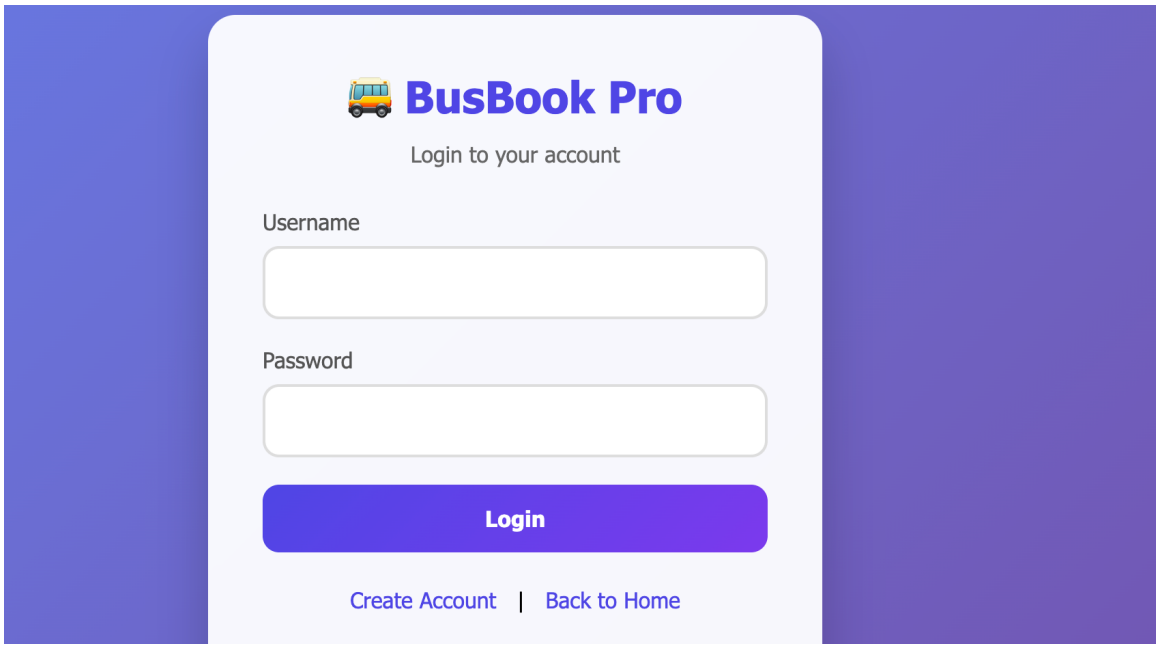


Figure 5: User Login Interface

The login page allows registered users to securely access their accounts by entering their username and password.

Figure 6: User Registration Page

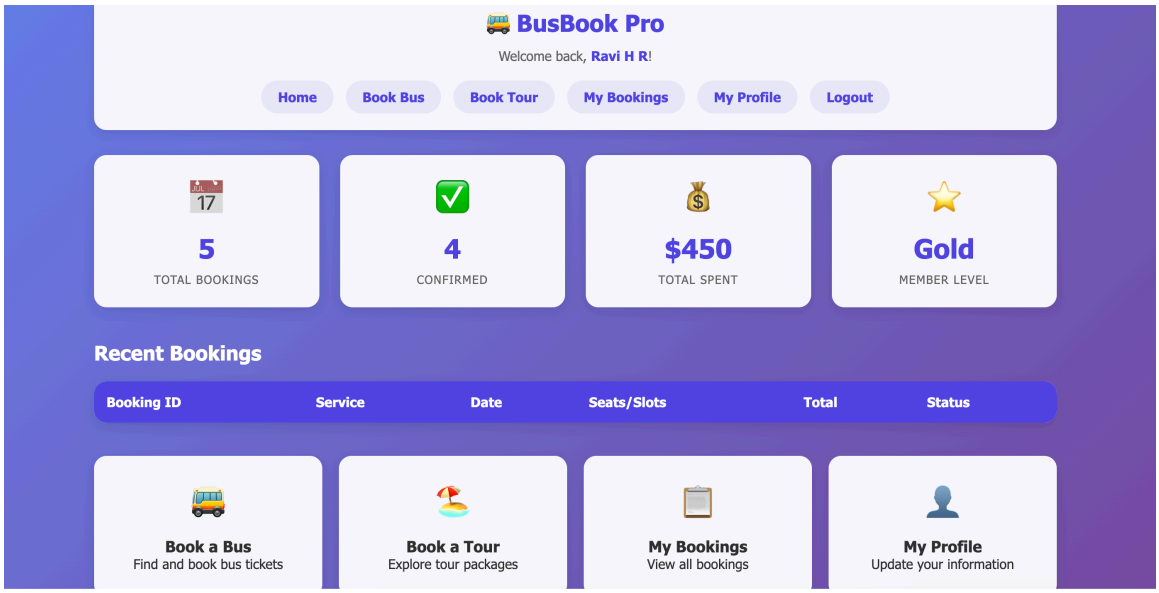


Figure 7: User Dashboard

The dashboard provides users with an overview of their recent booking and quick access to system feature such as viewing routes and tour packages.

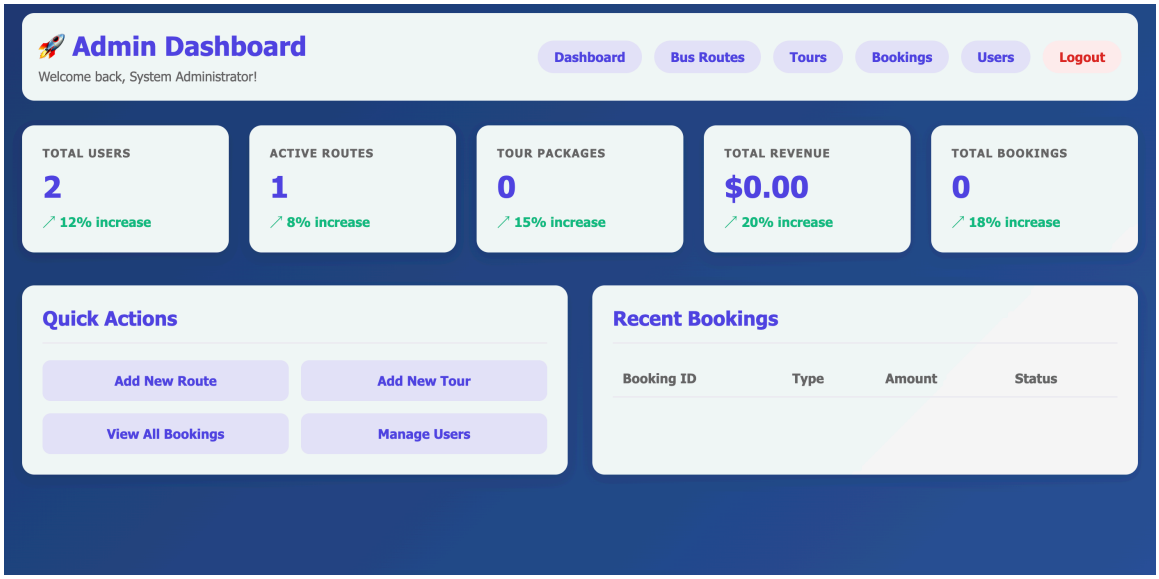


Figure 8: Admin Dashboard

The admin dashboard allows administrators to manage routes, tours, users, and booking records.

10. Testing and Quality Assurance

System testing was conducted for user registration, login, route browsing, tour browsing, booking functionality, and admin dashboard features.

Testing was performed to ensure that the system operates correctly and securely.

Test Type	**Description**
Unit Testing	Individual functions were tested
Integration Testing	Interaction between PHP and MySQL tested
System Testing	Complete booking workflow tested
Security Testing	Input validation and SQL injection protection tested

These tests ensured that the system behaves correctly and that user data remains protected.

Test Case	Description	Result
TC 1	User registration	Passed
TC 2	User login	Passed
TC 3	View routes	Passed
TC 4	Book transportation	Passed
TC 5	Book tours	Passed
TC 6	Admin dashboard access	Passed

The results indicate that all core functionalities operate as expected.

11. Project Risks

Potential risks include database failures, security vulnerabilities, and system downtime. These risks were mitigated through authentication, database validation, and structured system testing.

The below table explains the software development projects potential risks that often face:

Risk	Impact	Mitigation
Database errors	Data loss	Regular backups
Security vulnerabilities	Unauthorized access	Input validation
Development delays	Missing features	Prioritizing core functionality
Server downtime	Service disruption	Cloud hosting options

Managing these risks helps ensure that the system remains reliable and secure.

12. Lessons Learned

The development of this project provided valuable insights into full-stack web development and software engineering practices.

One of the main challenges was maintaining synchronization between seat availability and booking logic. Ensuring that users cannot book the same seat simultaneously required careful validation and database design.

Another challenge involved debugging backend logic and managing session states during user authentication.

Through the development process, important skills were gained in database normalization, modular programing, and system architecture design.

13. Future Improvements

Although the system meets its primary objectives, several enhancements can improve the system further.

Potential improvements include:

- Online payments gateway integration
- Automated email booking conformations
- Advanced analytics dashboard
- Mobile application development
- User review and rating system

These improvements would enhance both the functionality and scalability of the platform.

14. Conclusion

The transposition booking system demonstrates the successful implementation of web-based transport and tour booking application. The system integrates a responsive user interface, a robust backend, and a structured database to provide a reliable and secure booking platform.

By applying software engineering project such as requirements analysis, system architecture design, and testing, the project achieved its objectives of creating a functional and scalable booking system.

The experience gained from this project highlights the importance of structured development processes, documentation, and testing in building reliable software systems.

GitHub Link:

https://github.com/KruthikHR/Software_Engineering_Project